

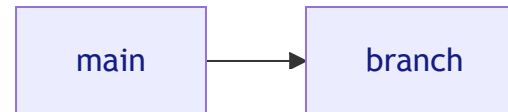
Herramientas de Desarrollo

Ingeniería de Sistemas

Universidad Austral — 2026

Dónde estamos

El ciclo de vida de un cambio:



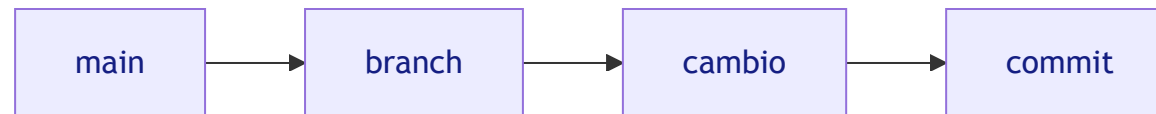
Dónde estamos

El ciclo de vida de un cambio:



Dónde estamos

El ciclo de vida de un cambio:



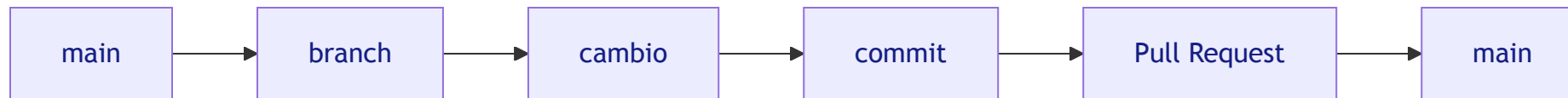
Dónde estamos

El ciclo de vida de un cambio:



Dónde estamos

El ciclo de vida de un cambio:



Hoy trabajamos sobre **todo lo que pasa antes del *merge***.

Automatizar

Las cuatro de hoy

Herramienta	Responde
<i>Formatter</i>	¿el código se ve como acordamos?
<i>Linter</i> / análisis estático	¿hay algo mal escrito?
<i>Coverage</i>	¿qué partes no ejercitan los tests?
<i>Git hooks</i>	¿cuándo corre todo lo anterior?

1 • *Formatter*

El problema

```
- fun calcularEnvio(peso:Double,distancia:Double,premium:Boolean):Double{  
-   var costo=peso*TARIFA_PESO+distancia*TARIFA_KM  
-   if(premium){costo=costo*DESCUENTO_PREMIUM}  
-   if(costo>LIMITE_ENVIO_GRATIS){return 0.0}  
-   return costo  
- }  
  
+ fun calcularEnvio(peso: Double, distancia: Double, premium: Boolean): Double {  
+     var costo = peso * TARIFA_PESO + distancia * TARIFA_KM  
+     if (premium) {  
+         costo = costo * DESCUENTO_PREMIUM  
+     }  
+     if (costo >= LIMITE_ENVIO_GRATIS) {  
+         return 0.0  
+     }  
+     return costo  
+ }
```

El mismo cambio, con el formato ya aplicado

```
fun calcularEnvio(peso: Double, distancia: Double, premium: Boolean): Double {  
    var costo = peso * TARIFA_PESO + distancia * TARIFA_KM  
    if (premium) {  
        costo = costo * DESCUENTO_PREMIUM  
    }  
-   if (costo > LIMITE_ENVIO_GRATIS) {  
+   if (costo >= LIMITE_ENVIO_GRATIS) {  
        return 0.0  
    }  
    return costo  
}
```

Qué hace un *formatter*

Reescribe el código para que cumpla un estilo. No opina sobre lo que el código hace: mueve espacios, saltos de línea, orden de *imports*.

```
fun main(){println( "hola" )} // antes
```

```
fun main() {                               // después
    println("hola")
}
```

Ya está elegido por vos

Lenguaje	Formatter
Kotlin	ktlint , <code>ktfmt</code>
Java	<code>google-java-format</code> , Spotless
JS / TS	Prettier, Biome
Python	Black, Ruff
Go	<code>gofmt</code> — viene con el lenguaje
Rust	<code>rustfmt</code> — viene con el lenguaje

ktlint en el *build*

En `build.gradle.kts`:

```
plugins {  
    kotlin("jvm") version "2.0.21"  
    id("org.jlleitschuh.gradle.ktlint") version "12.1.1"  
}
```

```
./gradlew ktlintCheck      # falla si algo no cumple  
./gradlew ktlintFormat     # lo arregla
```

check ya lo incluye

El plugin engancha `ktlintCheck` en la *task* `check`:

```
./gradlew check # compila, chequea formato, linter y tests
```

Una sola orden que siempre hace lo mismo.

.editorconfig

La configuración compartida, versionada con el proyecto:

```
[*.{kt,kts}]  
indent_size = 4  
max_line_length = 120  
ktlint_standard_no-wildcard-imports = disabled
```

Lo lee ktlint **y también el IDE.**

2 • *Linter* y análisis estático

Las convenciones del equipo

1. Clases en `PascalCase`
2. Funciones y variables en `camelCase`
3. Constantes en `UPPER_SNAKE_CASE`
4. Nombres descriptivos — nada de una sola letra
5. Sin *wildcard imports* (`import x.*`)
6. Sin *magic numbers* — todo valor literal va a una constante

¿Cuántos problemas de linteo ves?

```
fun calcular_total(precio: Double): Double {  
    return precio + IVA  
}
```

```
fun calcular_total(precio: Double): Double {  
    // ^^^^^^^^^^^^^^^ regla 2 – debería ser calcularTotal  
    return precio + IVA  
}
```

¿Cuántos problemas de linteo ves?

```
import java.util.*

class facturaService {
    val iva_default = 0.21

    fun calcular_total(p: Double) = p * (1 + iva_default)
}
```

```
import java.util.*           // 1 · regla 5 – wildcard import

class facturaService {       // 2 · regla 1 – FacturaService
    val iva_default = 0.21    // 3 · regla 2 – ivaDefault

    fun calcular_total(p: Double) = p * (1 + iva_default)
//                               ^ 4 · regla 4 – nombre de una letra
}
```

¿Cuántos problemas de linteo ves?

```
import java.util.*
import kotlin.math.*

class facturaService {
    val iva_default = 0.21

    fun calcular_total(p: Double): Double {
        if (p > 10000) return p * 0.9
        return p * (1 + iva_default)
    }
}
```

```
import java.util.*           // 1 · regla 5 – wildcard import
import kotlin.math.*         // 2 · regla 5 – wildcard import

class facturaService {      // 3 · regla 1 – FacturaService
    val iva_default = 0.21   // 4 · regla 2 – ivaDefault

    fun calcular_total(p: Double): Double {
        if (p > 10000) return p * 0.9
        //          ^^^^^ 5 · regla 6 – ¿qué es 10000?
        //          ^^ 6 · regla 6 – ¿y 0.9?
        return p * (1 + iva_default)
    }
}
```

¿Y acá?

```
import java.util.*
import kotlin.math.*

class facturaService {
    val iva_default = 0.21
    var Descuento = 0.0

    fun calcular_total(p: Double, q: Int, premium: Boolean): Double {
        var t = p * q
        if (t > 10000) {
            t = t * 0.9
        }
        if (premium) t = t - t * 0.15
        return t * (1 + iva_default)
    }

    fun VALIDAR(precio: Double) = precio > 0 && precio < 999999
}
```

```
import java.util.*           // 1 · regla 5 – wildcard import
import kotlin.math.*         // 2 · regla 5 – wildcard import

class facturaService {      // 3 · regla 1 – FacturaService
    val iva_default = 0.21   // 4 · regla 2 – ivaDefault
    var Descuento = 0.0      // 5 · regla 2 – descuento

    fun calcular_total(p: Double, q: Int, premium: Boolean): Double {
//      6 · regla 2 – calcularTotal · 7 · regla 4 – p → precio · 8 · q → cantidad
        var t = p * q         // 9 · regla 4 – t → total
        if (t > 10000) {      // 10 · regla 6 – ¿qué es 10000?
            t = t * 0.9       // 11 · regla 6 – ¿y 0.9?
        }
        if (premium) t = t - t * 0.15 // 12 · regla 6 – ¿y 0.15?
        return t * (1 + iva_default)
    }

    fun VALIDAR(precio: Double) = precio > 0 && precio < 999999
//      13 · regla 2 – validar · 14 · regla 6 – ¿y 999999?
}
```

Lo que encuentra un *linter*

```
$ ./gradlew detekt
```

```
> Task :detekt FAILED
```

```
facturaService.kt:4:7: Class and Object names should match the pattern: [A-Z][a-zA-Z0-9]* [ClassNameing]
```

```
facturaService.kt:7:9: Function names should match the pattern: [a-z][a-zA-Z0-9]* [FunctionNaming]
```

```
facturaService.kt:5:9: Variable names should match the pattern: [a-z][A-Za-z0-9]* [VariableNaming]
```

```
facturaService.kt:2:1: kotlin.math.* is a wildcard import. Replace it with fully qualified imports. [WildcardImport]
```

```
facturaService.kt:5:23: This expression contains a magic number. Consider defining it to a well named constant. [MagicNumber]
```

```
facturaService.kt:8:17: This expression contains a magic number. Consider defining it to a well named constant. [MagicNumber]
```

```
facturaService.kt:8:35: This expression contains a magic number. Consider defining it to a well named constant. [MagicNumber]
```

```
> Analysis failed with 7 weighted issues.
```

Detekt en el *build*

```
plugins {  
    id("io.gitlab.arturbosch.detekt") version "1.23.7"  
}  
  
detekt {  
    config.setFrom("$rootDir/config/detekt/detekt.yml")  
    buildUponDefaultConfig = true  
}
```

```
./gradlew detekt
```

detekt.yml

Se activan, se apagan y se calibran reglas:

```
complexity:  
  LongMethod:  
    active: true  
    threshold: 40      # el default casi nunca es tu número  
  TooManyFunctions:  
    active: false      # ← toda regla apagada lleva su motivo  
  
style:  
  MagicNumber:  
    ignoreNumbers: ['-1', '0', '1', '2']
```

Analizadores por lenguaje

Lenguaje	Herramienta
Kotlin	Detekt , ktlint
Java	Error Prone, SpotBugs, Checkstyle
JS / TS	ESLint, Biome
Python	Ruff, mypy, Pylint
Multi-lenguaje	SonarQube, CodeQL

3 • Coverage

JaCoCo en el *build*

```
plugins {  
    jacoco  
}  
  
tasks.test {  
    finalizedBy(tasks.jacocoTestReport)  
}
```

```
./gradlew test  
open build/reports/jacoco/test/html/index.html
```

Un umbral en el *build*

```
tasks.jacocoTestCoverageVerification {  
    violationRules {  
        rule {  
            limit { minimum = "0.80".toBigDecimal() }  
        }  
    }  
}  
  
tasks.check { dependsOn(tasks.jacocoTestCoverageVerification) }
```

4 • *Git hooks*

Antes de cada *commit*



```
./gradlew ktlintFormat  
./gradlew detekt  
./gradlew test jacocoTestReport  
git commit -m "feat: agregar validación de descuentos"
```

Qué son

Scripts que Git ejecuta solo cuando pasa algo en el repositorio.

```
$ ls .git/hooks/  
pre-commit.sample  commit-msg.sample  pre-push.sample
```

Si el *script* termina con **código distinto de 0**, la operación se cancela.

Un pre-commit real

```
#!/bin/sh
set -e

./gradlew ktlintCheck detekt --daemon
```

```
chmod +x .git/hooks/pre-commit
git commit -m "... " # ahora falla si el código no cumple
```

El problema: `.git/hooks` no se versiona

Un *hook* adentro de `.git/` existe **sólo en la máquina de quien lo escribió.**

Instalarlos desde el *build*

```
tasks.register<Copy>("installHooks") {  
    from("$rootDir/.githooks")  
    into("$rootDir/.git/hooks")  
    filePermissions { unix("0755") }  
}
```

Enganchada a un *build* que ya corren igual, la instalación **deja de depender de que alguien se acuerde.**

Y aun así, se pueden saltar

```
git commit --no-verify
```

El *hook* es **local y voluntario**.

Es una comodidad para quien escribe el código,
no una garantía para el equipo.

Lo mismo, en tres lugares

Dónde	Cuándo	Qué corre	¿Se puede evitar?
Editor	mientras escribís	<i>formatter, linter</i>	sí
<i>Hook</i>	al commitear	<code>ktlintCheck</code> , <code>detekt</code>	sí (<code>--no-verify</code>)
CI	al abrir el PR	<code>check</code> completo	no

Para el TP

- [] ktlint configurado, `.editorconfig` versionado
 - **2 reglas personalizadas** — configuración propia, distinta del default
- [] Detekt con `detekt.yml` propio
 - **2 reglas personalizadas** — activadas, apagadas o con otro *threshold*
- [] JaCoCo con reporte y umbral en `check`
 - **Coverage por encima del 80%**
- [] `pre-commit` versionado y una forma de instalarlo
- [] Todo enganchado a `./gradlew check`

Referencias

Repos de ejemplo:

[formatting-example](#) ·
[git-hooks](#)

Herramientas:

[ktlint](#) ·

[Detekt](#) ·

[JaCoCo](#) ·

[Git Hooks](#)